you can install the package *gdb* from its official repository. Windows users will have to rely on the MinGW variant.

Once you've compiled your program with the Debug flag, you can start GDB with your program loaded using the following command:

```
gdb PATH_TO_YOUR_PROGRAM
```

(The path can be relative; just the file name would do if you are in the same directory.)

Once started, GDB behaves like a command shell — it waits for your command, executes it and again waits. If you haven't given the program name with the *gdb* command, you can give it after GDB has started:

```
(gdb) file PATH_TO_YOUR_PROGRAM
```

(Please note that *(gdb)* is just a prompt, not part of the command you give).

run (shorthand: *r*) is the command to run your program. So all you have to do is type *r* and hit *Enter*. If everything goes well, you get your program's output, appended with GDB information on its termination (return values, premature ending, etc). You are then brought back to the GDB shell.

```
[Inferior 1 (process 7361) exited normally]
(gdb)
```

The above output shows that *Inferior 1*, our program, exits normally. If your program crashes somewhere in the middle, you are brought back to the GDB shell with an error message. Here you can perform a backtrack, monitor all the variables and registers, or restart the execution. If you want to have a similar monitoring by intentionally pausing at some particular points, you can set breakpoints. You can also press *Ctrl+C*, which might be a wild and inaccurate action, but don't do so unless you are stuck in an infinite loop or something like that. Consider the following code:

```
int main() {
        char *str = 0;
        puts(str);

        return 0;
}
```

And now see what happens in the GDB shell:

```
(gdb) r
Starting program: /tmp/a.out

Program received signal SIGSEGV, Segmentation fault.
__strlen_sse2 () at ../sysdeps/x86_64/multiarch/../
strlen.S:120
120 ../sysdeps/x86_64/multiarch/../strlen.S: No such file or
directory.
(gdb)
```

That doesn't make much sense, except that a segmentation fault has happened. Now you can give the backtrack *(bt)* command:

```
(gdb) bt
#0  __strlen_sse2 () at ../sysdeps/x86_64/multiarch/../
strlen.S:120
#1  0x00007ffff7a6d472 in _IO_puts (str=0x0) at ioputs.c:35
#2  0x0000555555554656 in main () at a.c:3
(gdb)
```

Now it is clear that it was caused by calling *puts()* with a NULL pointer, and it happened in *main ()* at *a.c:3*.

You can use the quit *(q)* command to exit from GDB. But quitting GDB is not necessary to go and make changes to your program. Once you come back, you can use *r* to restart your program. You can even use *make* to recompile your program (if you have a Makefile).

Perhaps the most useful command is *help (h)*, which is really informative. You can try *help breakpoints*, *help status*, etc. Try *help info* to learn about the useful *info (i)* command.

The GDB *print* command

You can use the *print (p)* command to get the value of a variable at a crash or breakpoint, if it is in the current scope (not in the above example since the crash happens in the library). You can also use the C member selection and de-referencing operators with it. For example, *p ptr* will print the address pointed by *ptr* while *p *ptr* will print the value pointed by it.

GDB manages to get detailed with structure members:

```
(gdb) p *token_new
$1 = {type = NAN_TOK_TYPE_PREPRODIR, token = {
    str = 0x7fff00000007 \, keyword = NAN_KW_LONG,
preprodir = NAN_PREPRODIR_IFNDEF,
    punctuator = NAN_PUN_ARROW, macro_argpos = 7}, where =
{
    file = 0x555555765ad0, line = 1, col =
8028058262076924005},
  global_str_literal_name = 0x0, next = 0x0}
```

Another advantage is the readability. In the above examples, many of the structure members have enum members assigned to them. GDB prints the human-readable names of them, where *printf* can only give you the integer values they represent.